

Practical: 2	Implementation and Time analysis of linear and binary search algorithm
19/08/2026	

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Algorithm:

Linear search algorithm:-

Input:- unsorted array
 output:- sorted array

Step 1:- START

Step 2:- Read the array elements 'A' and search.

Step 3:- set $i = 0$

Step 4:- Repeat while $i < n$

IF $A[i] = key$, display element for

Step 5:- increment $i = i + 1$
 If $i = n$, display element not found

Step 6:- STOP

Binary search algorithm

Input:- unsorted array
 output:- sorted Array

Step 1:- START

Step 2:- Read the sorted array 'A' and search element key

Step 3:- set low = 0, high = n-1

Step 4:- Repeat while low <= high

Step 5:- Find

$mid = (low + high) / 2$

Step 6:- If $A[mid] = key$, display "element found"

Step 7:- If $A[mid] < key$, set low = mid + 1

Step 8:- otherwise
 set high = mid - 1

Step 9:- If low > high, display "element not found"

Step 10:- STOP

Program:

```
#include <iostream>

#include <algorithm>

using namespace std;

// Linear Search
int linearSearch(int arr[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == key)
            return i;
    }

    return -1;
}

// Binary Search
int binarySearch(int arr[], int n, int key)
{
    int low = 0;
    int high = n - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (arr[mid] == key)

            return mid;
```

```
        else if (arr[mid] < key)
            low = mid + 1;

        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    int n, key;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter element to search: ";
    cin >> key;

    // Linear Search

    int linearResult = linearSearch(arr, n, key);
```

```
if (linearResult != -1)
    cout << "Linear Search: Element found at index "
        << linearResult << endl;
else
    cout << "Linear Search: Element not found" << endl;

// Sort array for Binary Search
sort(arr, arr + n);

cout << "Sorted array: ";
for (int i = 0; i < n; i++)
    cout << arr[i] << " ";

cout << endl;

// Binary Search
int binaryResult = binarySearch(arr, n, key);

if (binaryResult != -1)
    cout << "Binary Search: Element found at index "
        << binaryResult << endl;
else
    cout << "Binary Search: Element not found" << endl;

return 0;
}
```

Output:

```
Output
Enter number of elements: 5
Enter elements: 40 10 30 20 50
Enter element to search: 20
Linear Search: Element found at index 3
Sorted array: 10 20 30 40 50
Binary Search: Element found at index 1

=== Code Execution Successful ===
```

Time Complexity:

Linear Search:

Best Case:

Linear search:-
Best case:- element is found at '1'
for (i=0; i<n; i++) {
 if (arr[i] == key) {
 return i; → 1
 }
 return -1; → 1
}
T(n)=1
T(n)=O(1)

Average Case:

average case:-

```

for (i=0; i<n; i++) → n
{
  if (arr[i] == key) → 1
  {
    return i; → 1
  }
}
return -1; → 1
}
T(n) = O(n)
  
```

Worst Case:

Worst case:-

```

for (i=0; i<n; i++) → n
{
  if (arr[i] == key) → 1
  {
    return i; → 1
  }
}
return -1; → 1
}
T(n) = O(n)
  
```

Binary Search:

Best Case:

Element is found at the first position

```

for (i=0; i<n; i++) {
  if (arr[i] == key) {
    return i;
  }
}
T(n) = 1
T(n) = O(1)
  
```

Average Case:

```

low = 0; → 1
high = n-1; → 1
while (low ≤ high) { → 0 (log n)
  mid = (l+high)/2; → 1
  if arr[mid] == key → 1
  { return mid; }
  else if (arr[mid] < key) → 1
  {
    low = mid + 1; } → 1
  else {
    high = mid - 1; → 1
  }
}
return -1; → 1
T(n) = O(log n)
  
```

Worst Case:

```

low = 0; → 1
high = n-1; → 1
while (low ≤ high) { → 0 (log n)
  mid = (l+high)/2; → 1
  if arr[mid] == key → 1
  { return mid; }
  else if (arr[mid] < key) → 1
  {
    low = mid + 1; } → 1
  else {
    high = mid - 1; → 1
  }
}
return -1; → 1
T(n) = O(log n)
  
```

Conclusion:

From this implementation, I understood that the searching method depends on the type of data. Linear Search is simple and checks elements one by one, while Binary Search is faster because it repeatedly divides the sorted array into halves.